

Predictability is Trainable: Optimizing MoE LLMs to be Forecastable for Disk-Resident Inference

Spencer Churchill
Independent
spence@duck.com

Abstract

Inference engines such as Colibrì run 744B-parameter Mixture-of-Experts (MoE) models on consumer hardware by keeping the dense backbone resident and streaming expert weights from disk on demand — a reactive design whose throughput is bounded by fetch latency. Post-hoc predictors can forecast expert activations from earlier hidden states, but they chase a model never asked to be predictable. We ask whether predictability is *trainable*: we fold the accuracy of a small expert-activation predictor into the MoE’s own training loss, so the model is optimized jointly for quality and for being forecastable. At 341M-parameter scale we show that (i) predictability is a trainable, transferable **backbone property** — a fresh post-hoc predictor on the frozen jointly-trained backbone recovers the full gain (+6–7 pts top- k hit rate over a linear control, +2.9–5.2 pts over a ranking-aware control; both 3 seeds); (ii) the gain is **structural, not sharpening** — entropy penalties at any strength cannot reach it at matched entropy; (iii) it **grows with training** while its quality cost vanishes; and (iv) it converts to systems value (+3.2% tok/s, –31% misprefetch waste in a disk-queue-accurate simulator; +5.0% mean tok/s in a purpose-built real engine). Finally we map the boundary: a LoRA fine-tune on pretrained OLMoE-1B-7B transfers only a small fraction of the effect even at $10\times$ loss pressure, indicating the method is **pretraining-time only**.

1 Introduction

Sparse MoE LLMs activate only a small fraction of parameters per token, which makes their *weights* — not their compute — the binding resource on consumer hardware. Engines like Colibrì [10] exploit this: the dense backbone stays resident while tens of thousands of routed experts are streamed from disk, cached, and pinned by learned hot-stores. Every such engine is reactive: the router fires, and only then does the fetch begin. Colibrì’s own mitigation is a router-lookahead thread (PILOT) that prefetches next-layer experts, exploiting that routing is $\sim 72\%$ predictable one layer ahead on a production model.

A literature of post-hoc predictors (§2) shows expert activations can be forecast from earlier hidden states with 93–97% accuracy. But all of it treats the model as fixed: the predictor chases the model. We invert the relationship — **the accuracy of an independent expert-activation predictor becomes an explicit term in the MoE’s own training loss** — so the backbone is regularized toward being forecastable at multi-layer lookahead horizons while the predictor simultaneously adapts to the model.¹

Contributions, each matched to an exhibit:

- **C1 — Predictability is a trainable, transferable backbone property.** Fresh post-hoc predictors trained on the frozen jointly-trained backbone recover the entire gain; an engine needs no co-training to exploit it (Table 1).
- **C2 — The gain is structural, not sharpening.** Entropy-penalty controls at any strength cannot reach the jointly-trained predictability at matched entropy; $\sim +2$ –2.4 pts of the advantage is pure structure (Figure 1).
- **C3 — The effect is robust and grows with training.** It survives init + data-order variance (± 0.3 pts) and *increases* at $4\times$ tokens, where the quality cost flips to a small gain (Table 2).

¹Code, data pipeline, run scripts, and the full audit trail: <https://github.com/splch/predictability-aware-training>

- **C4 — The accuracy converts to systems value, with a mapped boundary.** +3.2% tok/s and −31% misprefetch waste over the post-hoc control at Colibrì-like geometry (+1.6%/−27% at ranking-control accuracies), gated by $\text{fetch_time} \leq \text{compute_window}$; +5.0% mean tok/s in a real O.DIRECT engine; and a LoRA fine-tune on a pretrained model recovers only a fraction of the effect even at $10\times$ loss pressure — the method is pretraining-time only (Table 4).

2 Related Work

Post-hoc expert prediction (frozen model). Pre-attention linear predictors with ranking-aware losses reach 93–97.6% top- k accuracy [22]; cross-layer gate inputs enable edge offloading with 99% cache hits (Fate [5]); trace-trained sequence predictors raise cache hit rates from 17% to 72% at 10% budget (MoE-Beyond [7]); PreScope [19] adds globally scheduled prefetching. Apple’s LLM-in-a-Flash [1], PowerInfer-2 [18], and MoE-Infinity [17] address adjacent offloading/caching problems. All of these train predictors *against* a fixed model; none change the model.

Training-time routing modification. StickyMoE [11] adds a differentiable routing-consistency loss for temporal reuse — the closest philosophy to ours, but it optimizes *stickiness* (token t vs $t-1$), not *forecastability from earlier layers*; we show in §4.6 that it does not buy lookahead predictability in our reimplementation. Oracle-MoE [21] redesigns the routing input for emergent locality (no explicit objective). ReMoE [23] fine-tunes only the router post-hoc. Pre-gated MoE [9] is the closest co-design: a pre-gate module that *replaces* next-layer routing, co-trained end-to-end with the backbone — but with no predictor-accuracy objective and no realized-router to evaluate forecasts against. Halfway Speculative Decoding [14] applies the same co-design pattern (train the target to be draftable) in token space. Routing regularization for quality or stability [4, 8, 12] never targets inference-time predictability. No prior work, to our knowledge, adds the accuracy of an *independent* expert-activation predictor as an explicit term in the MoE’s own loss at multi-layer lookahead; our stronger claim is empirical: that predictability is a transferable backbone property (C1) with a structural signature distinguishable from entropy reduction (C2).

Our novelty statement, precisely scoped. Prior systems work trains predictors post-hoc against frozen models; prior training-time work optimizes routing temporal locality; Pre-gated MoE co-trains a lookahead gate that *replaces* routing. We are aware of no work that (a) makes an independent predictor’s accuracy an explicit training objective of the MoE itself, and (b) demonstrates the resulting predictability is a recoverable property of the backbone’s representations. Where this paper’s ranking-aware control reimplements [22]’s protocol, we say so; all comparisons use our implementation, not their published numbers.

3 Method

Model. GPT-style decoder ($d=512$, 12 layers, 16 experts, top-2, 341M total / 99M active parameters), trained on FineWeb-edu [15] (GPT-2 BPE [16], document-shuffled stream with EOS, ctx 512, bf16 autocast).

Predictor. Per-(layer, horizon) heads from the pre-MoE hidden state at layer l to the router decision at layer $l+h$, $h \in \{1, 2, 4\}$. Two families: *linear* (single projection) and *ranking-MLP* (2-layer MLP with a margin loss: the weakest true top- k logit must exceed the strongest intruder by a margin), the latter reimplementing the protocol of [22].

Joint loss. $L = L_{\text{LM}} + 0.01 \cdot L_{\text{balance}} + \lambda \cdot L_{\text{pred}}$ (+ optional entropy or sticky terms for controls). L_{pred} is a soft cross-entropy of predictor logits against the realized top- k set (uniform $1/k$ mass). Gradients from L_{pred} reach the backbone only through hidden representations (realized top- k targets are argmax, non-differentiable) — the loss shapes representations, not router weights directly. The load-balancing term follows Switch Transformer [6].

Isolation protocol (a first-class methodological contribution). To test whether predictability lives in the *backbone* rather than in predictor co-adaptation, we freeze a trained backbone and train a *fresh* predictor on it (2000–3000 steps), comparing against the same fresh-predictor protocol on the baseline backbone. This mirrors deployment: an inference engine trains its own predictor against a shipped model.

Table 1: hit@ k (between-seed std), fresh linear post-hoc predictor. Chance = 0.125.

backbone	$h=1$	$h=2$	$h=4$	val LM
baseline	0.826 (0.003)	0.797 (0.002)	0.732 (0.004)	5.767 (0.006)
joint ($\lambda=0.3$)	0.888 (0.002)	0.865 (0.002)	0.796 (0.004)	5.790 (0.006)

Table 2: Robustness of the joint advantage across variance axes and training length.

	+hit@ k vs control ($h=1$ / $h=4$)	quality cost
25M tokens, init-only variance	$+6.2 \pm 0.4$ / $+6.4 \pm 0.6$	$+0.023 \pm 0.009$ nats
25M tokens, init+data variance	$+6.2 \pm 0.3$ / $+6.8 \pm 0.3$	$+0.033 \pm 0.016$ nats
100M tokens (4$\times$)	+6.1 / +7.9	−0.019 nats (free)

Metrics. hit@ k per horizon (top- k set overlap $|\text{pred} \cap \text{true}|/k$); router entropy; adjacent-layer routing persistence; utilization max; val LM loss on a fixed 16-batch held-out set drawn once before training and shared by every run; zero-shot downstream accuracy (HellaSwag [20], ARC [3], PIQA [2]) scored by length-normalized continuation logprob. Unless stated, numbers are $n=3$ seeds (init + data order) with between-seed std.

4 Experiments

4.1 Setup and main result (Table 1)

Baseline ($\lambda=0$) vs joint ($\lambda=0.3$), 25M tokens, horizons $\{1, 2, 4\}$. Controls are fresh post-hoc predictors on the frozen baseline backbone; the joint row is the fresh-predictor-on-joint-backbone isolation configuration.

Under the stronger ranking-MLP control (3 seeds): baseline 0.903 (0.001) / 0.865 (0.002) / 0.795 (0.001) vs joint **0.933 (0.002) / 0.907 (0.004) / 0.847 (0.008)** — the advantage compresses to **$+2.9 \pm 0.2$ / $+4.3 \pm 0.3$ / $+5.2 \pm 0.8$ pts** (10–25 $\times$ seed noise) but survives intact under an identical predictor. We report +6–7 (linear) and +3–5 (ranking) as dual honest effect sizes throughout. Paired quality cost: $+0.023 \pm 0.009$ nats; $+0.033 \pm 0.016$ with data-order variance included.

4.2 The isolation test (C1)

A fresh post-hoc predictor on the frozen joint backbone recovers the co-trained predictor’s accuracy *exactly* (0.890/0.866/0.796 vs 0.887/0.863/0.793). The predictability is a property of the weights — the deployable artifact.

4.3 Structure, not sharpening (C2, Figure 1)

An entropy-penalty ladder ($\lambda_{\text{ent}} \in \{0.005, 0.01, 0.05\}$) plus the baseline and joint points, all evaluated with the ranking-MLP probe:

hit@1 vs entropy: baseline (2.26, 0.903) < $\lambda 0.005$ (1.62, 0.912) < $\lambda 0.01$ (1.17, 0.916) < **joint (2.10, 0.930)**; near-collapse (0.28) reaches only 0.883 under the linear probe. Interpolating to the joint model’s own entropy, sharpening alone yields ~ 0.906 – 0.910 : **$\sim +2.0$ – 2.4 pts of the joint advantage is pure structure** — routing that is simultaneously informative and forecastable, not merely peaked.

4.4 Robustness and growth with training (C3, Table 2)

At 4 $\times$ training the advantage *grows* ($h4$ +7.9) and the quality cost disappears; baseline routers sharpen naturally with training (entropy 2.26 $\rightarrow$ 1.98) and the joint model stays ahead. Zero-shot downstream accuracy (HellaSwag/ARC-Easy/ARC-Challenge/PIQA, $n=1000$ each) is at chance for both arms at 25M tokens (the model is too undertrained for task sensitivity; paired deltas -0.6 ± 1.0 pp across 12 seed $\times$ benchmark pairs); at 100M tokens the joint model is at or above baseline on all four benchmarks.

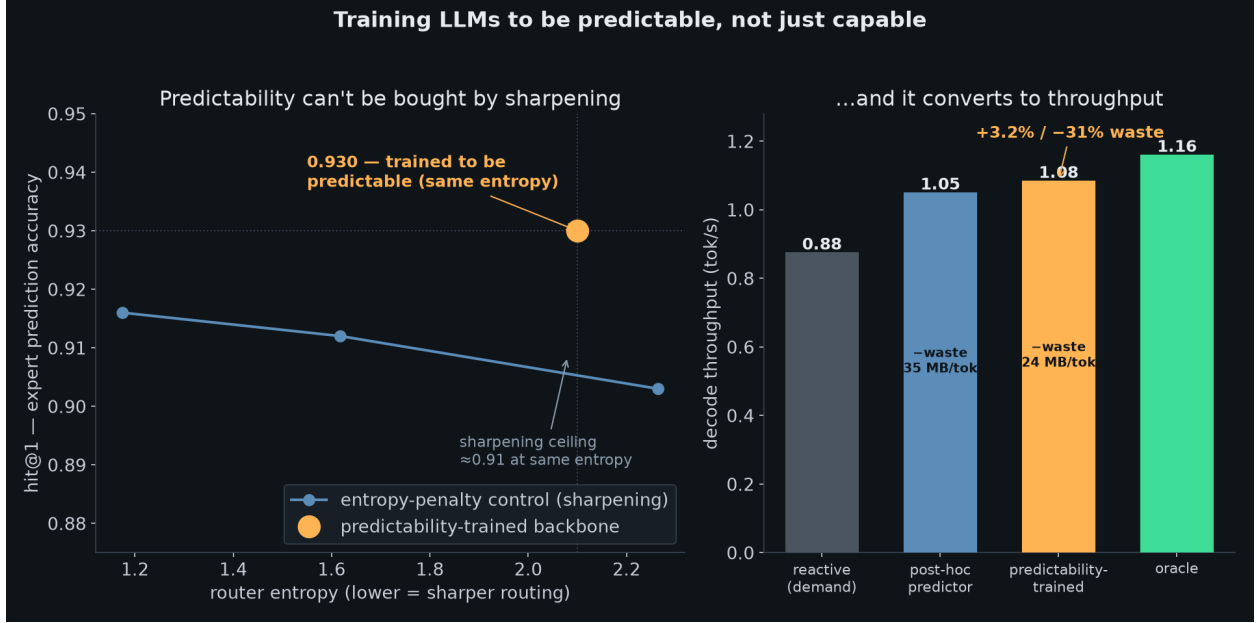


Figure 1: (left) The entropy-penalty frontier plateaus at ~ 0.91 hit@1 while the predictability-trained backbone reaches 0.930 at matched entropy; (right) decode throughput and misprefetch waste at Colibrì-like geometry (corrected simulator).

Table 3: StickyMoE-style routing-consistency sweep (post-hoc linear probe on each backbone). The consistency loss monotonically *reduces* lookahead predictability while raising router entropy.

arm	$h=1$	$h=2$	$h=4$	entropy	val LM
baseline	0.822	0.791	0.724	2.264	5.949
sticky 0.3	0.808	0.774	0.707	2.485	5.945
sticky 1.0	0.793	0.754	0.682	2.599	5.957
sticky 3.0	0.774	0.734	0.658	2.676	5.976

4.5 Degeneracy checks

At $\lambda=0.3$: entropy -9% (no collapse), adjacent-layer persistence $\approx$ chance (no temporal copying), utilization unchanged, token-ID $\rightarrow$ expert lookup probe equal in both backbones (no memorization), marginal expert usage near-uniform (popularity trivial predictor hits 0.15). At $\lambda=1.0$ the degenerate mode appears measurably: entropy -28% and $+0.62$ nats quality cost — the frontier turns, visible in our diagnostics.

4.6 Training-time baseline: StickyMoE (Table 3)

A routing-consistency loss (our reimplementation of StickyMoE’s objective) *monotonically reduces* lookahead hit@ k while raising router entropy, and does not stack with our loss (0.864 vs 0.884 joint-only). Scope: we tested only lookahead predictability; StickyMoE’s actual target — temporal-reuse cache locality — is orthogonal and untested here.

5 Systems Analysis

Simulator. A trace-driven engine with an honest single-disk FIFO queue (demand-priority; prefetch issued only into idle compute windows that fit the fetch; bandwidth conservation is exact). Synthetic Colibrì geometry: 75×256 experts, top-8, 19 MB int4 experts, 4 ms/layer dense compute, $C=2000$ -expert cache, Zipf+locality routing, predictor accuracies measured in §4.

Table 4: Tier C isolation test on OLMoE-1B-7B (hit@ k at $h=1/h=2/h=4$). At $\lambda=0.1$ the fine-tune leaves backbone predictability unchanged; at $10\times$ pressure a small transfer appears, still far weaker than pretraining.

post-hoc probe	on baseline backbone	on joint ($\lambda=0.1$)	on joint ($\lambda=1.0$)
linear	0.799 / 0.781 / 0.752	0.800 / 0.783 / 0.755	0.812 / 0.799 / 0.778
ranking-MLP (177M)	0.845 / 0.801 / 0.739	0.845 / 0.801 / 0.741	0.847 / 0.807 / 0.753

Throughput (prefetch $h=1$). demand 0.876 $\rightarrow$ base-control 1.051 $\rightarrow$ joint 1.085 $\rightarrow$ oracle 1.161 tok/s (linear-control accuracies): **+3.2% joint vs base, -31% misprefetch waste (24.0 vs 34.6 MB/tok)**, $\sim 31\%$ of the base-to-oracle headroom captured. At ranking-control accuracies (0.903 vs 0.933, 3-seed): **+1.6% and -27% waste** — the conversion paired with the honest effect size. TTFT (cold cache): -11% from prediction generally, -1-3% joint vs base — not the differentiator.

Gating condition. Across a 3×3 latency/bandwidth grid the joint-vs-base uplift is stable (+2.9-3.2%) whenever prefetch fires; below ~ 5 GB/s (19 MB expert), fetch time exceeds the compute window and *no* prefetch is possible: **bandwidth, not latency, gates the regime** — a design constraint for engines exploiting trained predictability.

Real engine. A purpose-built O.DIRECT disk-resident engine for the toy model (192 experts $\times$ 2.88 MB, demand-priority disk queue, post-hoc predictor driving a prefetch thread, cache budget $C=64$): joint vs base backbone, deployable configuration, 6 decode sequences: **+5.0% mean tok/s (range -7% to +14.5%)**, consistent with the simulator’s direction and magnitude; per-sequence variance is large at toy scale, so we treat this as qualitative confirmation and keep the quantitative claim on the simulator.

6 Boundary: the method is pretraining-time (Table 4)

OLMoE-1B-7B [13] (64 experts top-8, 3T pretraining tokens), LoRA r16 on attention+experts plus trainable routers, $\lambda \in \{0.1, 1.0\}$, 25M tokens:

At the lower pressure ($\lambda=0.1$) the isolation test is a tie under both probes. At $10\times$ pressure ($\lambda=1.0$) a small backbone effect appears — +1.3/+1.8/+2.6 pts under the linear probe, +0.2/+0.6/+1.4 under the ranking probe, growing with horizon — at +0.011 nats and with router entropy flat (3.719 vs 3.748, so not sharpening). But this remains **2.5-5 $\times$ weaker than the pretraining effect under the linear probe** (and more under the ranking probe, whose ceiling leaves less room): the boundary is dose-dependent, not a strict zero, and the pretraining regime is qualitatively different. We read this as **boundary mapping, not failure**: representations must be shaped while plastic; the entire training-time prior-art family (StickyMoE, Oracle-MoE) shares the property. Scope of the negative: one seed, one LoRA rank, one token count (two λ values); heavier fine-tuning (unfrozen blocks, full FT) is untested. Note also the baseline itself: a post-hoc predictor on stock OLMoE reaches 0.845 $h=1$ — post-hoc predictors are strong on real models, and that is the bar any pretraining application must clear.

7 Limitations

Positive evidence is one toy from-scratch model (341M, 16 experts, FineWeb-edu only, ≤ 100 M tokens); systems numbers are simulator + toy-engine, not a production engine; quality is val-LM plus chance-level zero-shot downstream probes (no consistent joint deficit at 25M tokens; joint $\geq$ baseline on all four benchmarks at 100M tokens) — task sensitivity at this scale is limited; $n=3$; the ranking control is our reimplement of [22]’s protocol; StickyMoE was tested on our metric only; the Tier C boundary rests on one seed / LoRA rank / token count (two λ values). At toy $B=8/C=64$ the joint engine row slightly exceeds the oracle (97.9 vs 97.4 tok/s) — a queue-interaction effect consistent with the waste ordering (oracle issues more prefetch traffic), noted for completeness.

8 Conclusion

Predictability is trainable: an MoE optimized jointly for output quality and for the accuracy of an independent expert-activation predictor becomes measurably, structurally, and transferably more forecastable — and the gain converts to throughput and waste reductions wherever an engine has disk slack. The effect must be baked in during pretraining. For pretraining labs targeting disk-resident deployment, predictability pressure is a cheap ($\lambda \approx 0.3$), quality-neutral auxiliary objective; for engine authors, the result defines what a shipped model can make recoverable. Data, code, and the full audit trail (including three red-team rounds and all negative results) are available in the repository: <https://github.com/splch/predictability-aware-training>.

References

- [1] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. *arXiv:2312.11514*, 2023.
- [2] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of AAAI*, 2020. *arXiv:1911.11641*.
- [3] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. In *arXiv:1803.05457*, 2018.
- [4] Damai Dai, Li Dong, Shuming Ma, Bo Zheng, Zhifang Sui, Baobao Chang, and Furu Wei. Stablemoe: Stable routing strategy for mixture of experts. *arXiv:2204.08396*, 2022.
- [5] Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. *arXiv:2502.12224*, 2025.
- [6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv:2101.03961*, 2021.
- [7] Nishant Gavhane, Arush Mehrotra, Rohit Chawla, and Peter Proenca. Moe-beyond: Learning-based expert activation prediction on edge devices. *arXiv:2508.17137*, 2025.
- [8] Rizhen Hu, Yuan Cao, Boao Kong, Mou Sun, and Kun Yuan. Synergistic intra- and cross-layer regularization losses for MoE expert specialization. *arXiv:2602.14159*, 2026.
- [9] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *51st International Symposium on Computer Architecture (ISCA)*, 2024. *arXiv:2308.12066*.
- [10] JustVugg. Colibrì: Run frontier MoE models on hardware you own. <https://github.com/JustVugg/colibri>, 2026. Pure-C disk-resident MoE inference engine; PILOT router-lookahead thread.
- [11] Ali Kayyam. Sticky routing: Training MoE models for memory-efficient inference. *arXiv:2607.08780*, 2026. Method referred to as StickyMoE.
- [12] Ang Lv, Jin Ma, Yiyuan Ma, and Siyuan Qiao. Coupling experts and routers in mixture-of-experts via an auxiliary loss. *arXiv:2512.23447*, 2025.
- [13] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, et al. OLMoE: Open mixture-of-experts language models. *arXiv:2409.02060*, 2024.
- [14] Marian-Sergiu Nistor. Halfway speculative decoding: Direct acceptance rate optimization with joint drafter-target training. OpenReview, <https://openreview.net/forum?id=NLdAp0Oizb>, 2026.

- [15] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv:2406.17557*, 2024.
- [16] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.
- [17] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv:2401.14361*, 2024. USENIX ATC 2025.
- [18] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv:2406.06282*, 2024.
- [19] Enda Yu, Dezun Dong, Zhaoning Zhang, Zhe Bai, Weiling Yang, Haojie Wang, Dongsheng Li, Yongwei Wu, and Xiangke Liao. Layerscope: Predictive cross-layer scheduling for efficient multi-batch MoE inference on legacy servers. *arXiv:2509.23638*, 2025. ICS 2026; the prediction-driven scheduling system is named PreScope.
- [20] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of ACL*, 2019. *arXiv:1905.07830*.
- [21] Jixian Zhou, Fang Dong, Ruijun Huang, Hengjie Cao, Mengyi Chen, Yifeng Yang, Anrui Chen, Mingzhi Dong, Yujiang Wang, Dongsheng Li, David A. Clifton, Qin Lv, Rui Zhu, Chun Zhang, Fan Yang, Tun Lu, Ning Gu, and Li Shang. Oracle-MoE: Locality-preserving routing in the oracle space for memory-constrained large language model inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.
- [22] Shien Zhu, Samuel Bohl, Robin Oester, and Gustavo Alonso. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv:2511.10676*, 2025.
- [23] Xiongwei Zhu, Xiaojian Liao, Tianyang Jiang, Yusen Zhang, Liang Wang, and Limin Xiao. Remoe: Boosting expert reuse through router fine-tuning in memory-constrained MoE LLM inference. *arXiv:2605.27081*, 2026. ICML 2026.